A Numerical Benchmark for the Merton Problem under Corrected GBM Drift: Finite Differences, PINNs, and Terminal-Loss Deep BSDEs

Arnav Yadav
VIT Bhopal University
arnav.24bai10392@vitbhopal.ac.in

Somya Patel
VIT Bhopal University
somya.24bsa10058@vitbhopal.ac.in

October 10, 2026

Abstract

We compare four numerical methods for the Merton portfolio problem using equity parameters estimated from Apple adjusted closing prices: a log-wealth finite-difference grid, two value-network formulations, and a forward deep BSDE. The sample covers 23 May 2023 through 22 May 2026 and contains 752 daily log returns. The annualized mean log return is $m = 0.201649$, and the estimated GBM volatility is $\sigma = 0.256573$. Accordingly, the drift in $\mathrm{d}S/S$ is $\mu = m + \sigma^2/2 = 0.234564$, rather than m. With constant risk-free rate $r = 0.0368$, relative risk aversion $\gamma = 5$, and no intermediate consumption, the classical Merton allocation is $\pi^\star = 0.600837$.

On a shared evaluation mesh of 10^4 points, the 500×252 log-wealth grid achieves a policy mean absolute error (MAE) of 2.52×10^{-4}. Across five runs, the power-scaled value network, whose free function $g(t, \log w)$ multiplies the CRRA profile, has policy MAE 1.62×10^{-3} (sample standard deviation 5.8×10^{-4}) and value MAE 7.09×10^{-4}. The more restrictive ansatz $g = g(t)$ recovers the optimal weight to round-off; its mean value error is 4.64×10^{-4}. Across five runs, the forward deep BSDE has policy MAE 0.0215 (standard deviation 0.0072), while its estimate of $V(0, 1)$ differs from the closed-form value by about 5×10^{-5}. On 20,000 shared paths, the certainty equivalent for each one-dimensional method is within 10^{-4} of the closed-form value, 1.10105. Adding an Ornstein–Uhlenbeck factor correlated with the stock lowers the grid weight at $(t, Y) = (0, 0)$ from 0.601 to 0.499. The power-scaled network has policy error 0.018 against this factor grid, compared with about 0.097 for the deep BSDE. Under the constraint $0 \le \pi \le 0.4$, the grid's weight differs from $\pi^\star$ by 0.201 on average, and the network matches the constrained grid to 10^{-8}. All reported values are taken from the run ledger; the experiment driver is `run_one.py`.

Keywords: Merton problem; GBM drift; finite differences; physics-informed neural networks; deep BSDE.

1 Scope and contributions

This paper presents a controlled numerical comparison; it does not establish a new theorem or make claims about problems at dimension $d = 100$. The control problem follows Merton [1969] and Merton [1971]. The value-network approaches draw on Raissi et al. [2019] and Sirignano and Spiliopoulos [2018]. We cite E et al. [2017] and Han et al. [2018] for the forward discretization, but train the terminal-loss objective $\mathbb{E}[(Y_T - U(W_T))^2] - \mathbb{E}[U(W_T)]$, not the distinct martingale objective $-\mathbb{E}[Y_0] + \mathrm{Var}(Y_0)$.

We apply the GBM drift correction before training and report five independently seeded runs for each one-dimensional network under a prespecified budget. We also examine finite-difference convergence as the wealth grid is refined. Because the CRRA-shaped ansatz implies the optimal policy analytically, we treat it as a value-function approximation rather than evidence of policy learning. Following the stochastic-opportunity-set setting of Kim and Omberg [1996], we solve a one-factor problem on a (t, Y) grid: CRRA homogeneity separates wealth, while the correlated factor remains a state variable. We evaluate the binding leverage constraint against the constrained grid rather than the unconstrained Merton weight.

The study does not propose a high-dimensional solver or an existence result for a consumption–investment problem. Nor does the small policy error of the CRRA ansatz show that a network has learned the optimal allocation. The risk-free rate is held fixed, and the empirical calibration uses a single equity.

2 Market inputs and calibration

Let S_t denote Apple's adjusted closing price. The 753 trading sessions from 23 May 2023 through 22 May 2026 yield $n = 752$ daily log returns for AAPL, obtained using `yfinance`. Using 252 trading days per year,

$$m = 252 \cdot \frac{1}{n} \sum_{i=1}^{n} \log \frac{S_i}{S_{i-1}} = 0.20164940530246397, \tag{1}$$

$$\sigma = \sqrt{252}\, s_{n-1}(\log S_i / S_{i-1}) = 0.2565727586666484. \tag{2}$$

If $\mathrm{d}S/S = \mu\,\mathrm{d}t + \sigma\,\mathrm{d}B$, then $\mathbb{E}[\log(S_{t+\Delta}/S_t)] = (\mu - \sigma^2/2)\Delta t$, so the drift that enters the wealth equation is

$$\mu = m + \sigma^2/2 = 0.23456419554737107. \tag{3}$$

An independent `yfinance` download for the same period reproduced m and σ to within 10^{-7}. The experiments use the frozen estimates reported above. The risk-free rate $r = 0.0368$ is a specified constant, not an estimate of the Treasury-bill rate. We set relative risk aversion to $\gamma = 5$, the horizon to $T = 1$, and initial wealth to $W_0 = 1$. No intermediate consumption is included in the one-dimensional model.

Table 1: Quantities that are not interchangeable. m is the mean log return. μ is the GBM drift.

Quantity	Value
m	0.201649405302464
$\sigma^2/2$	0.032914790244907
$\mu = m + \sigma^2/2$	0.234564195547371
σ	0.256572758666648
r	0.0368
γ	5
$\pi^\star = (\mu - r)/(\gamma\sigma^2)$	0.600836870221195
$V(0,1)$	−0.170138511234813

Using m in place of the GBM drift would change the optimal weight from 0.600837 to 0.500837. We do not rescore checkpoints trained with that substitution.

3 The Merton portfolio problem

Wealth invested in the risky asset and the money-market account evolves according to

$$\mathrm{d}W_t = W_t(r + \pi_t(\mu - r))\,\mathrm{d}t + W_t\pi_t\sigma\,\mathrm{d}B_t. \tag{4}$$

The investor maximizes expected terminal utility, $\mathbb{E}[U(W_T)]$, where

$$U(w) = \frac{w^{1-\gamma}}{1-\gamma}, \qquad \gamma > 1. \tag{5}$$

The value function $V(t,w)$ satisfies the Hamilton–Jacobi–Bellman equation derived in Merton [1969],

$$V_t + \sup_{\pi}\left\{w(r + \pi(\mu - r))V_w + \tfrac{1}{2}\pi^2\sigma^2 w^2 V_{ww}\right\} = 0, \tag{6}$$

subject to $V(T,w) = U(w)$. The optimal allocation and value function are

$$\pi^\star = \frac{\mu - r}{\gamma\sigma^2}, \tag{7}$$

$$V(t,w) = \mathrm{e}^{A(T-t)}\frac{w^{1-\gamma}}{1-\gamma}, \tag{8}$$

$$A = (1-\gamma)\left(r + \frac{(\mu-r)^2}{2\gamma\sigma^2}\right). \tag{9}$$

For the frozen coefficients, $\pi^\star = 0.600836870221195$ and $V(0,1) = -0.170138511234813$.

4 Numerical methods

4.1 Finite differences

We discretize the PDE in log wealth, $x = \log w$, over $x \in [-2.5, 4.5]$. Time stepping is fully implicit ($\theta = 1$), and we use an upwind approximation for the advection term. At each grid node, we compute the candidate control from the first-order condition

$$\pi = -\frac{(\mu - r)V_x}{\sigma^2(V_{xx} - V_x)}. \tag{10}$$

We then project it onto the admissible interval: $[-1, 5]$ in the unconstrained experiments and $[0, 0.4]$ in the constrained experiments. Non-finite first-order conditions are set to zero; the analytical weight $\pi^\star$ is never inserted. The reference grid for the main tables has $N_x = 500$ wealth points and $N_t = 252$ time steps.

4.2 Collocation and the terminal condition

At each Adam iteration, both value networks draw a fresh Latin-hypercube sample. Interior collocation points are sampled jointly in time and wealth, with time uniform on $[0, T]$ and wealth uniform on $[0.5, 2]$. Each one-dimensional run uses 25,000 interior points. The terminal-condition loss uses 8,000 wealth samples from the same interval, with time fixed at T and target $U(w)$. The sampler seed is determined by the run seed and epoch, so repeating a command with the same seed reproduces the minibatches on the same device type.

Following the collocation approach of Raissi et al. [2019], we use a network with a power head that scales its output to the utility function. Five hyperbolic-tangent layers, each of width 128, map $(\log w, t)$ to a scalar g. We define the value function as

$$V(t, w) = \mathrm{e}^{g(t, \log w)} \frac{w^{1-\gamma}}{1-\gamma}. \tag{11}$$

Because g can depend on wealth, this architecture does not impose $V_w/(wV_{ww}) = 1/\gamma$. The network does not output a policy directly; after training, we recover the weight by applying the first-order condition to automatic derivatives of V. The loss combines the mean-square HJB residual, ten times the relative terminal mismatch, and penalties equal to 0.1 times the squared positive parts of V_{ww} and $-V_w$. For unconstrained runs, the residual uses the reduced Hamiltonian. For capped runs, it evaluates the Hamiltonian at the control projected onto $[0, 0.4]$. We did not use an unscaled output network because target values span several orders of magnitude over the wealth interval $[0.5, 2]$. The power head addresses this scale variation without imposing $\pi^\star$.

4.3 The CRRA ansatz

The CRRA-shaped network is defined by

$$V(t,w) = \mathrm{e}^{g_\theta(t)} \frac{w^{1-\gamma}}{1-\gamma}. \tag{12}$$

For any g_θ, the ratio $V_w/(wV_{ww})$ equals $1/\gamma$. Thus, when the training coefficients match the evaluation coefficients, the recovered weight equals $\pi^\star$ up to machine precision. A policy error of approximately 10^{-8} therefore checks the automatic derivatives; it does not measure policy learning. The network trains the scalar function g_θ, and we report its value-function error.

4.4 Forward deep BSDE

Our scheme uses the forward discretization described by Han et al. [2018] and E et al. [2017]. We evolve a scalar initial value Y_0 and a network Z alongside controlled wealth; the model has no intermediate consumption. Under a Markov control that satisfies the HJB equation, the value process is a martingale, giving the update

$$Y \leftarrow Y + Z(t,W)\,\Delta B. \tag{13}$$

We evaluate Z at the wealth value before applying the current Brownian increment. The policy network affects the state evolution through the wealth process W, and a $2\tanh$ transformation restricts its output to $(-2,2)$. Each training batch contains 1,024 paths with 64 time steps, and the loss is

$$\mathbb{E}\big[(Y_T - U(W_T))^2\big] - \mathbb{E}[U(W_T)]. \tag{14}$$

The mean-square term trains (Y_0, Z), while the expected-utility term guides policy optimization. A flexible Z can fit the terminal condition even when the control is poor, so terminal matching alone is insufficient. We initialize Y_0 at $U(W_0)$ rather than at its closed-form value. This terminal-loss objective differs from the martingale approach, which imposes $Y_T = U(W_T)$ and minimizes $-\mathbb{E}[Y_0] + \mathrm{Var}(Y_0)$.

4.5 Training and evaluation protocol

We train each one-dimensional network for 2,000 epochs using Adam with learning rate 10^{-3} and retain the checkpoint with the lowest training loss. The power-scaled network, CRRA ansatz, and deep BSDE each use seeds $\{0,1,2,3,4\}$. We evaluate all one-dimensional methods on the same 10^4-point mesh (seed 12345), with log-uniform wealth on $[0.5,2]$ and uniform time on $[0,1)$. Traded utility is estimated on the same 20,000 paths for every method, using 252 steps and seed 7. Certainty-equivalent wealth is defined by $U(w) = \mathbb{E}[U(W_T)]$. Training used CUDA; CPU runs may differ slightly in floating-point values, but the seeds and grid acceptance threshold are unchanged.

5 Results for the constant-coefficient problem

5.1 Finite-difference grid

Experiment E1 uses the corrected drift μ on a grid with 500 wealth points and 252 time steps. On the shared evaluation mesh, the policy MAE is 2.5217×10^{-4}. The near equality of the root-mean-square and mean absolute errors indicates that the grid's small policy bias is nearly constant across the mesh, as expected when the optimal weight itself is constant. At the grid node nearest $(t, W) = (0, 1)$, the computed value is -0.169453, compared with the exact value -0.170139. The value-function MAE over the mesh is 4.398×10^{-3}. On the common paths, both the closed-form and grid policies yield certainty-equivalent wealth of 1.10105 to five decimal places; their mean utility is -0.170103.

Table 2: Finite-difference refinement study (E2). Policy error decreases as the number of wealth points N_x increases; adding time steps does not change the recovered policy.

N_x	N_t	policy MAE	value MAE
128	64	3.878×10^{-3}	1.620×10^{-2}
128	128	3.878×10^{-3}	1.641×10^{-2}
128	252	3.878×10^{-3}	1.652×10^{-2}
256	64	9.649×10^{-4}	8.223×10^{-3}
256	128	9.649×10^{-4}	8.413×10^{-3}
256	252	9.649×10^{-4}	8.506×10^{-3}
500	64	2.522×10^{-4}	4.134×10^{-3}
500	128	2.522×10^{-4}	4.311×10^{-3}
500	252	2.522×10^{-4}	4.398×10^{-3}

When we go from 128 to 500 wealth points (roughly four times as many), the policy error drops by a factor of about fifteen. That is close to the improvement you would expect from a second-order method on a smooth solution. Adding more time steps does not reduce the value error on the random mesh. The policy, which is the quantity that is actually traded, is already fixed by the wealth grid.

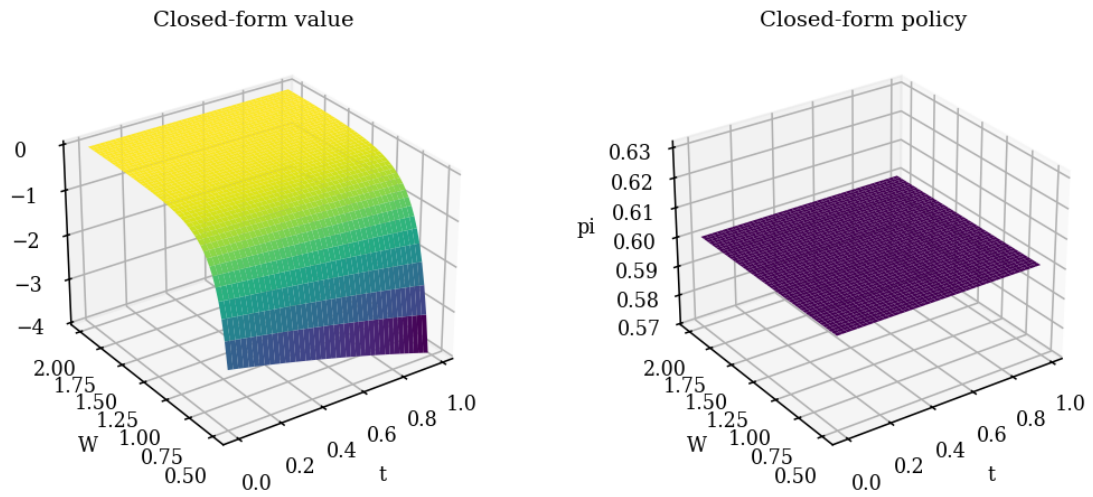


Figure 1: Closed-form value and the constant weight $\pi^\star$ on $(t, W) \in [0, 0.98] \times [0.5, 2]$.

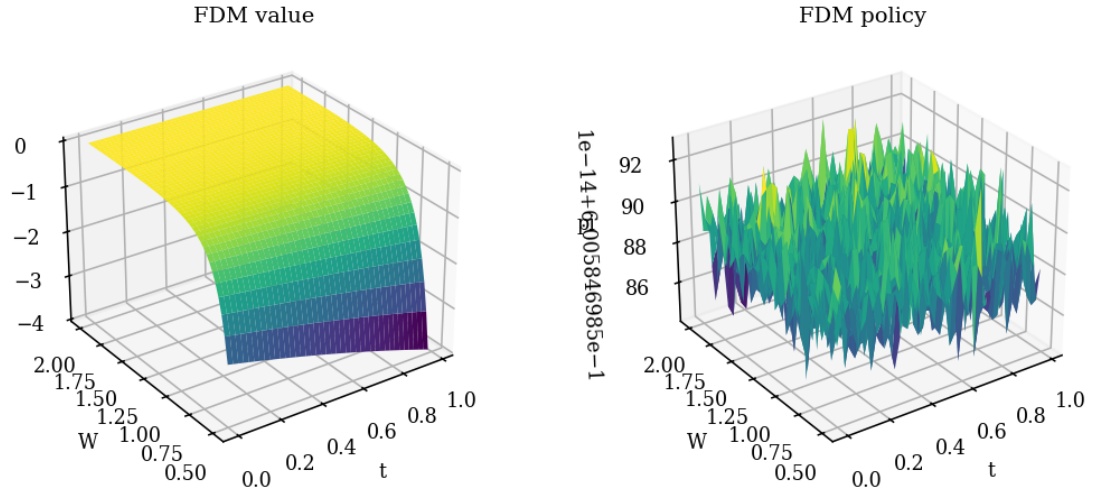


Figure 2: E1 finite-difference value and policy on the same window. The policy surface is flat at $\pi^\star$ up to the 2.5×10^{-4} bias in Table 2.

5.2 Sensitivity to risk aversion and volatility

E2, reported above, changes the number of grid points and keeps γ and σ at the Apple values. E6 does the opposite. It keeps one grid, $N_x = 256$ and $N_t = 128$, and changes the investor. The two risk-aversion levels are $\gamma = 3$ and $\gamma = 7$. Volatility is the Apple estimate scaled by 0.8 and by 1.2. Each row uses the corrected drift $\mu = m + \sigma^2/2$ for that row's σ. Policy MAE ranges from 3.4×10^{-4} to 1.7×10^{-3}. The grid is not tuned only to $(\gamma, \sigma) = (5, 0.256573)$.

Table 3: Sensitivity results (E6). Each row reports a separate solve with its own optimal weight $\pi^\star$; policy error is measured against the corresponding closed-form solution.

γ	σ	$\pi^\star$	policy MAE
3	0.256573	1.001395	3.353×10^{-4}
7	0.256573	0.429169	1.659×10^{-3}
5	0.205258	0.938808	1.508×10^{-3}
5	0.307887	0.417248	6.701×10^{-4}

5.3 CRRA ansatz: five-seed results

In E4, we train the time-only CRRA ansatz for 2,000 epochs. Across seeds, policy MAE ranges from 3.18×10^{-8} to 3.26×10^{-8}. This near-zero error reflects the architecture's analytical structure and serves as a derivative check, rather than an independent measure of policy performance. Across five seeds, value MAE averages 4.64×10^{-4}, with sample standard deviation 1.81×10^{-4}. The individual value errors are 4.61, 3.02, 6.42, 6.49, and 2.66, each in units of 10^{-4}. On the shared paths, the ansatz yields the same certainty equivalent as the closed-form policy, 1.10105, as expected when both use $\pi^\star$.

5.4 Power-scaled value network

Experiment E3 trains the network $V = \mathrm{e}^{g(t,\log w)} w^{1-\gamma}/(1-\gamma)$ for 2,000 epochs using five seeds, 25,000 collocation points, and 8,000 terminal points. The seed-level policy MAEs are 1.81×10^{-3}, 2.41×10^{-3}, 1.46×10^{-3}, 8.14×10^{-4}, and 1.60×10^{-3}. Their mean is 1.62×10^{-3}, with sample standard deviation 5.79×10^{-4}. Value MAE averages 7.09×10^{-4}, with standard deviation 1.60×10^{-4}; this is slightly higher than for the time-only ansatz. Unlike that ansatz, the power-scaled network does not impose the optimal weight through its architecture. Its policy error is nevertheless only several times the grid error of 2.52×10^{-4}. On the shared paths, its traded certainty equivalent averages 1.1010493, compared with 1.1010499 for the closed-form policy.

5.5 Deep BSDE: five-seed results

Experiment E5 trains five seeds for 2,000 epochs, using batches of 1,024 paths and 64 time steps. The policy MAEs are 0.03019, 0.01840, 0.02672, 0.01184, and 0.02021, with mean 0.02147 and sample standard deviation 0.00720. The mean absolute gap $|Y_0 - V(0,1)|$ is 4.85×10^{-5} (standard deviation 3.83×10^{-5}). The traded certainty equivalent averages 1.100984, approximately 6.5×10^{-5} below the closed-form result. Thus, the BSDE estimates the initial value accurately, while its mesh-based policy is less accurate than the grid and power-scaled network; both results are summarized in Table 4.

Table 4: Mean errors on the one-dimensional evaluation mesh. The CRRA policy error is a derivative check; sample standard deviations are reported in the reproducibility section. The closed-form certainty equivalent on the shared paths is 1.101050. The traded certainty equivalents are 1.101050 for the grid, 1.101050 for the ansatz, 1.101049 for the power-scaled network, and 1.100984 for the deep BSDE.

Method	seeds	policy MAE	value MAE	$\lvert Y_0 - V(0,1)\rvert$
FDM 500×252	1	2.52×10^{-4}	4.40×10^{-3}	—
CRRA $g(t)$	5	3.2×10^{-8}	4.6×10^{-4}	—
Power $g(t, \log w)$	5	1.6×10^{-3}	7.1×10^{-4}	—
Deep BSDE	5	2.1×10^{-2}	—	4.9×10^{-5}

6 Stochastic investment opportunity set

With constant coefficients, CRRA homogeneity reduces the problem to a PDE with one spatial state variable. Adding another risky asset with constant coefficients would not introduce a state-dependent investment opportunity set, so we instead add a stochastic factor, following the setting of Kim and Omberg [1996]. Let the factor satisfy the Ornstein–Uhlenbeck equation

$$\mathrm{d}Y_t = -\kappa Y_t \,\mathrm{d}t + \nu \,\mathrm{d}B_t^Y, \tag{15}$$

with $\kappa = 1.2$, $\nu = 0.30$, and $\mathrm{Corr}(\mathrm{d}B, \mathrm{d}B^Y) = \rho = 0.45$. The stock drift is $\mu + \phi Y$, where $\phi = 0.20$. The value function still separates in wealth,

$$V(t, w, y) = f(t, y) \frac{w^{1-\gamma}}{1-\gamma}, \tag{16}$$

although the optimal allocation now includes a hedging demand:

$$\pi^\star(t, y) = \frac{\mu + \phi y - r}{\gamma \sigma^2} + \frac{\rho \nu}{\sigma \gamma} \frac{f_y}{f}. \tag{17}$$

The function f varies with y and is computed on a grid of 81 factor nodes over $[-1.5, 1.5]$ and 200 time steps. The policy is lagged by one time level in the linear implicit step, and Neumann boundary conditions are imposed at both factor-domain endpoints. As a validation, setting $\phi = 0$ and $\rho = 0$ recovers e^{AT} with maximum absolute error 2.52×10^{-4}. This check confirms that the factor solver reproduces the constant-opportunity benchmark without inserting its solution.

The factor value network represents f as $f = \mathrm{e}^{g(t,y)}$, using 4,096 Latin-hypercube collocation points and a terminal penalty enforcing $g(T, y) = 0$. The factor BSDE uses two Brownian increments and evaluates both components of Z before updating the state; its objective is the same terminal-loss-plus-utility objective. Experiments E7 and E8 use seeds 0 and 1, respectively, and train for 800 epochs. Both methods are evaluated against the factor grid on 4,000 points, rather than against the constant-coefficient weight $\pi^\star$.

Using the Apple calibration, the factor grid gives $\pi(0, 0) = 0.49918$ and $V(0, 1, 0) = -0.16217$.

For comparison, the corresponding constant-opportunity values are 0.60084 and −0.17014. The change in allocation reflects both the state-dependent drift and the hedging demand; the optimal weight is no longer constant.

For E7 and E8, the factor network and two-noise deep BSDE are each trained for 800 epochs. The power-scaled network's policy MAEs against the factor grid are 0.01810 (seed 0) and 0.01861 (seed 1); the corresponding value MAEs are 0.0437 and 0.0451. The deep BSDE policy MAEs are 0.0949 and 0.0994. Its initial value estimate is approximately −0.1717, compared with the grid value −0.1622 at $(W, Y) = (1, 0)$, a difference of 9.6×10^{-3}. In this experiment, the power-scaled network has lower reported policy error than the deep BSDE. Figure 8 shows the grid policy over (t, Y).

Table 5: Factor-model results compared with the (t, Y) grid. Each network is trained for 800 epochs using two seeds. The grid values $\pi(0, 0)$ and $V(0, 1, 0)$ are seed-independent.

	seed 0	seed 1
Power-head policy MAE	0.01810	0.01861
Power-head value MAE	0.04368	0.04511
Deep BSDE policy MAE	0.09488	0.09940
$\lvert Y_0 - V_{\text{grid}}(0, 1, 0)\rvert$	0.00957	0.00954

7 Results under a binding portfolio constraint

The unconstrained Merton weight is approximately 0.60, so the interval $[0, 1]$ does not bind. Experiments E9 and E10 instead impose the tighter constraint $\pi \in [0, 0.4]$. With constant coefficients, the constrained optimum is the projection of $\pi^\star$ onto this interval. We use the finite-difference solution under the same constraint as the network's reference. The power-scaled value network is trained for 2,000 epochs with seeds 0 and 1, and the Hamiltonian is evaluated at the projected control. A network that returned the unconstrained weight $\pi^\star$ would therefore be incorrect.

Experiments E9 and E10 use a 400×200 grid and network seeds 0 and 1. On the shared mesh, the constrained grid's policy differs from the unconstrained $\pi^\star$ by MAE 0.200837, equal to $\pi^\star - 0.4$ because the upper bound binds throughout. The power-scaled network reproduces the constrained grid policy to within 6.0×10^{-9} for both seeds. Its value MAEs against the grid are 4.12×10^{-3} (seed 0) and 4.15×10^{-3} (seed 1). At the grid node nearest $(t, W) = (0, 1)$, the constrained value is −0.166096, greater (less negative) than the unconstrained value −0.170139. This increase reflects the value of the leverage excluded by the cap. The network does not return the unconstrained weight.

8 Figures

The figures show training behavior, recovered policies, and pointwise errors. Each image is generated from a logged checkpoint.

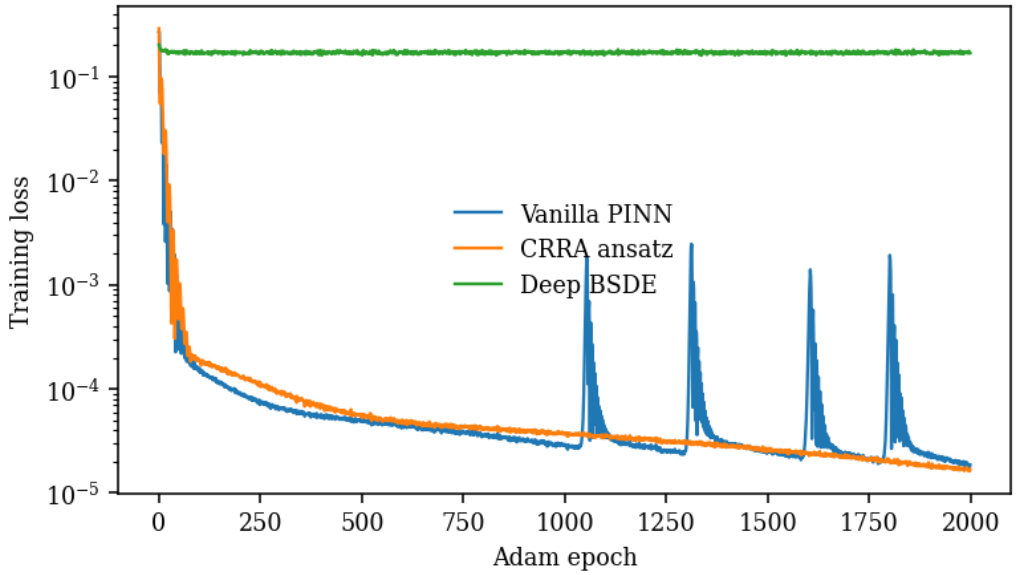


Figure 3: Training loss by epoch for seed 0 of the power-scaled value network, the time-only CRRA ansatz, and the forward deep BSDE. The vertical axis is logarithmic. For each method, the reported checkpoint is the epoch with the lowest loss, which need not be the final epoch. The losses generally decline through most of training, and the CRRA ansatz reaches the lowest loss among the three methods.

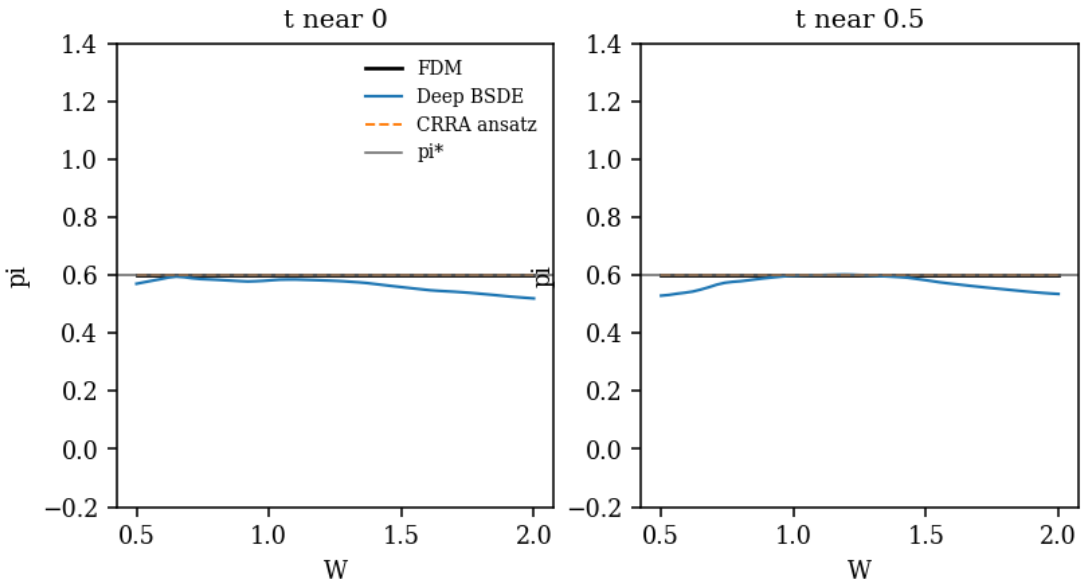


Figure 4: Portfolio weight π as a function of wealth W at two fixed times for the finite-difference grid (E1), the CRRA ansatz, and deep BSDE seed 0. The horizontal line marks the closed-form weight $\pi^\star \approx 0.601$. The left panel uses a narrow vertical range around this value; the right panel shows a wider range. The grid and ansatz policies nearly coincide with $\pi^\star$. The deep BSDE policy is more variable and mildly wealth-dependent, consistent with its larger policy MAE.

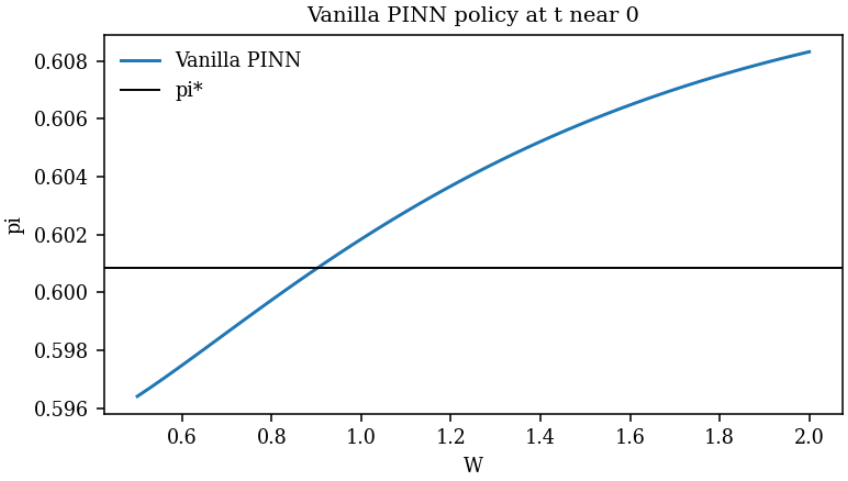


Figure 5: Seed-0 policy from the power-scaled value network near $t = 0$, plotted against wealth. The recovered policy is nearly flat and close to $\pi^\star$, with small deviations near the edges of the wealth interval.

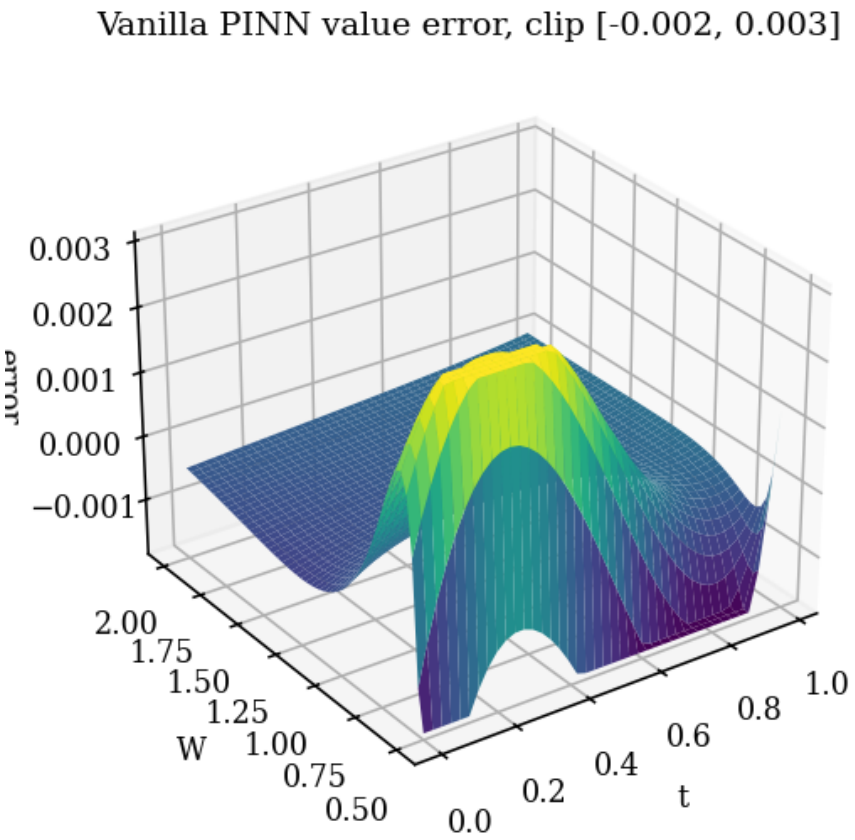


Figure 6: Pointwise error of the seed-0 power-scaled value network relative to the closed-form solution, shown over time and wealth. For visualization, the surface is clipped at the 2nd and 98th error percentiles, -1.78×10^{-3} and 3.03×10^{-3}. Absolute errors are below 10^{-3} over most of the domain, with the largest deviations near the training-window corners.

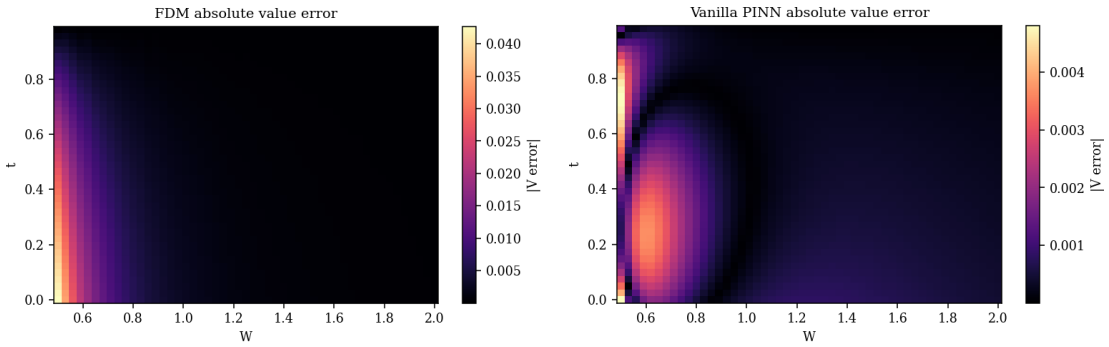


Figure 7: Absolute value-error heat maps for the 500×252 finite-difference grid (E1, left) and the seed-0 power-scaled network (right). Each panel uses its own color scale to show the structure of the errors. Over wealth $[0.5, 2]$, the network's absolute value error is smaller than the grid's, although the grid policy error is already only 2.5×10^{-4}.

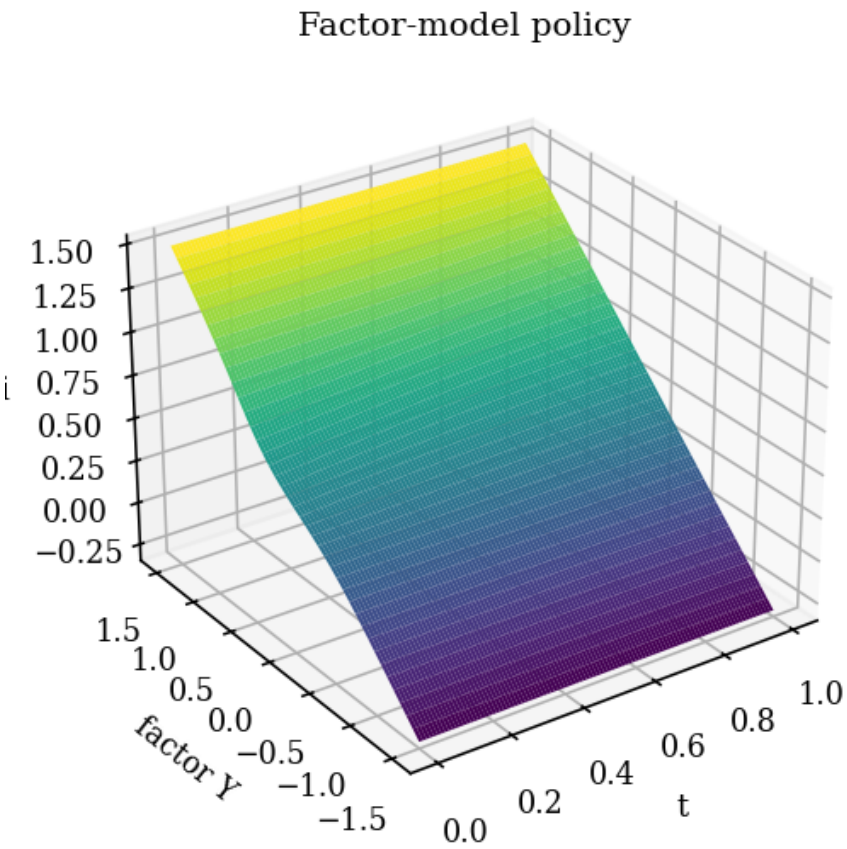


Figure 8: Optimal risky weight on the (t, Y) grid for the one-factor problem (E7). The Ornstein–Uhlenbeck factor Y is correlated with the stock. At $Y = 0$, the weight is approximately 0.50, compared with 0.60 in the constant-opportunity model. The surface illustrates how the allocation varies with the factor and time.

9 Reproducibility

Code and run artifacts are available in the companion GitHub repository. The repository's `README.md` describes the environment and provides instructions for reproducing experiments E1–E10, regenerating the figures, and building this PDF. Experiment E1 stops if policy MAE exceeds 2×10^{-3} or if the absolute error in $V(0, 1)$ exceeds 5×10^{-3}. Table 4 prints means only. The sample standard deviations are in `results/v2/figures/table_main.csv`: 5.79×10^{-4} for the power-scaled policy and 7.20×10^{-3} for the deep BSDE policy.

10 Limitations and scope

The risk-free rate is held fixed rather than estimated from the price data. The calibration uses one equity, and the factor extension has only one additional state; neither experiment tests a genuinely high-dimensional problem. Because the CRRA ansatz imposes the constant-coefficient policy by construction, its small policy error does not show that a general value network has learned the optimal allocation. The deep BSDE is trained with 64 time steps but evaluated using

wealth paths simulated with 252 steps, so its initial-value estimate Y_0 need not coincide with the utility estimate from the trading simulation. Finally, the constrained constant-coefficient problem has a closed-form projection solution. We include the grid as a benchmark for future constraints without such an explicit solution and to avoid supplying $\pi^\star$ to the network.

References

Weinan E, Jiequn Han, and Arnulf Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380, 2017. doi: 10.1007/s40304-017-0117-6.

Jiequn Han, Arnulf Jentzen, and Weinan E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.

Tong Suk Kim and Edward Omberg. Dynamic nonmyopic portfolio behavior. *The Review of Financial Studies*, 9(1):141–161, 1996. doi: 10.1093/rfs/9.1.141.

Robert C. Merton. Lifetime portfolio selection under uncertainty: The continuous-time case. *The Review of Economics and Statistics*, 51(3):247–257, 1969. doi: 10.2307/1926560.

Robert C. Merton. Optimum consumption and portfolio rules in a continuous-time model. *Journal of Economic Theory*, 3(4):373–413, 1971. doi: 10.1016/0022-0531(71)90038-X.

Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

Justin Sirignano and Konstantinos Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018. doi: 10.1016/j.jcp.2018.08.029.